Polynomial Curve Fitting (Part 2)

Peter Adepoju
(My study note on the topic)

1 Generalization and Model Selection

Now that we have successfully trained our model, by using the Normal Equation to find the specific weights $\mathbf{w}^*$ that minimize the sum of squared errors on our training data. A novice might look at the training error, see that it is very low (perhaps even zero), and conclude that the model is perfect. This is a dangerous illusion. The true goal of machine learning is not to minimize error on data we already have, but to accurately predict data we have *never seen before*.

1.1 The Generalization Gap

1.1.1 Memorization vs. Learning

Imagine a student preparing for a calculus exam. They have a list of 10 practice problems with answers.

- **Student A (Memorization):** Memorizes the answer to every specific practice problem. If the exam consists of those exact 10 questions, they will score 100%. If the exam has new questions, they will fail.
- **Student B (Learning):** Learns the underlying rules of differentiation. They might make small mistakes on the practice problems, but they can solve new problems they have never seen.

In our context:

- The **Training Set** is the practice exam (the 10 blue dots).
- The **Test Set** is the final exam (future data).
- **Generalization** is the ability of the model (Student B) to perform well on the Test Set.

1.1.2 Defining the Gap

We formally define two error metrics:

1. **Training Error (E_{train}):** The average error calculated on the data used to fit the parameters $\mathbf{w}$.
2. **Test Error (E_{test}):** The average error calculated on a separate, independent dataset.

The **Generalization Gap** is the difference between these two metrics:

$$\text{Gap} \approx E_{test} - E_{train} \tag{1}$$

Complexity: As we increase the complexity of our model (the polynomial degree M), the Training Error E_{train} will *always* decrease. A polynomial of degree $N-1$ can pass through every single one of N data points perfectly, resulting in $E_{train} = 0$. However, as E_{train} goes to zero, E_{test} often explodes to infinity. The model has "memorized" the noise in the training data rather than learning the signal. This phenomenon is known as **Overfitting** (Vapnik, 1998).

1.1.3 The Fundamental Assumption of ML

Why do we expect a model trained on past data to work on future data at all? We rely on the **i.i.d. assumption** (Independent and Identically Distributed):

- The training data and the test data are generated by the **same underlying process** (the same probability distribution).
- Each data point is independent of the others.

If this assumption holds, a simple model that captures the underlying structure (the signal) should have a small Generalization Gap. A complex model that captures the noise will have a large gap.

1.2 Time-Series Splitting

In standard machine learning problems (like classifying images of cats and dogs), the order of the data does not matter. The first image is independent of the second. Therefore, we typically *shuffle* the dataset randomly before splitting it into training and testing sets to ensure both sets have a representative distribution of classes.

However, the Mauna Loa dataset is a **Time Series**. The CO_2 level in 1990 is strongly correlated with the level in 1989. This temporal dependence imposes a strict constraint on how we evaluate our models.

1.2.1 The Prohibition of Random Shuffling

The Golden Rule of Forecasting: Never use the future to predict the past.

If we were to randomly shuffle the Mauna Loa data, our training set might contain points from the year 2020, while our test set contains points from 1960.

- The model would "see" the high CO_2 levels of 2020 during training.
- It would effectively "memorize" that the trend goes up.
- When asked to predict 1960 (in the test set), it would use this future knowledge to interpolate backwards.

This is a form of *Data Leakage* known as **Look-Ahead Bias**. It leads to overly optimistic performance estimates that collapse when the model is deployed in the real world, where we sadly cannot peek at next year's data (Hyndman & Athanasopoulos, 2018).

1.2.2 Chronological Splitting

To simulate a realistic forecasting scenario, we must split the data **chronologically**. We draw vertical lines on the timeline:

1. **Past (Training):** The oldest data. Used to fit the weights $\mathbf{w}$.
2. **Recent Past (Validation):** Data immediately following the training set. Used to select the best polynomial degree M.
3. **Future (Test):** The most recent data. Locked away in a "vault" and used **only once** for the final report.

1.2.3 Python Implementation

We implement this by calculating integer indices that correspond to the cut-off points. In the example code, we use a 70% / 15% / 15% split.

```python
N = len(x)  # Total number of data points (e.g., 815)

# Calculate cut-off indices
train_end = int(0.70 * N)        # Index for 70% mark
val_end   = int(0.85 * N)        # Index for 85% mark (70% + 15%)

# 1. Create the Training Set (First 70%)
x_train = x[:train_end]
t_train = t[:train_end]

# 2. Create the Validation Set (Next 15%)
x_val   = x[train_end:val_end]
t_val   = t[train_end:val_end]

# 3. Create the Test Set (Last 15%)
x_test  = x[val_end:]
t_test  = t[val_end:]
```

Listing 1: Chronological Splitting

Array Slicing: We use Python's powerful **Slicing Syntax** `[start:stop]`.

- `[:train_end]` means "from the beginning (index 0) up to, but *not including*, `train_end`".
- `[train_end:val_end]` means "start exactly at `train_end` and go up to `val_end`".
- `[val_end:]` means "start at `val_end` and go all the way to the end".

This ensures that there is **zero overlap** between the sets and that the order is strictly preserved.

1.2.4 Sanity Check

After splitting, it is good practice to verify the shapes to ensure no data was lost.

```python
print(f"Total N: {N}")
print(f"Train: {len(x_train)}")
print(f"Val:   {len(x_val)}")
print(f"Test:  {len(x_test)}")
```

Mathematically, we must ensure:

$$N_{train} + N_{val} + N_{test} = N$$

Any discrepancy here would indicate a bug in the index calculation (e.g., off-by-one errors due to rounding).

1.3 Root Mean Square Error (RMSE)

While the Sum of Squared Errors (SSE) is mathematically convenient for differentiation, it has two major flaws as a reporting metric:

1. **Scale Dependence:** The SSE grows with the number of data points N. A dataset with 1000 points will naturally have a much higher SSE than a dataset with 10 points, even if the model's accuracy is identical. This makes it impossible to compare the Training set (large N) with the Test set (small N).
2. **Unit Confusion:** If the target t is measured in "ppm" (parts per million), the SSE is measured in "ppm^2". This physical unit is meaningless to a human interpreter.

To address these issues, we use the **Root Mean Square Error (RMSE)**.

1.3.1 The Mathematical Definition

The RMSE normalizes the error by the number of data points and returns the value to the original units.

$$E_{RMSE} = \sqrt{\frac{1}{N}\sum_{n=1}^{N}(y_n - t_n)^2} = \sqrt{\frac{2E(\mathbf{w})}{N}} \tag{2}$$

Interpretation: The RMSE represents the **standard deviation of the residuals**. It tells us, on average, how far the model's predictions deviate from the true values.

- If RMSE = 2.5, it means our predictions are typically off by ±2.5 ppm.
- This allows us to make intuitive statements like "The model is accurate to within 2.5 ppm."

1.3.2 Python Implementation

In Python, we calculate RMSE using NumPy's vectorized operations. This approach avoids explicit loops and is highly optimized.

```python
import numpy as np

def rmse(y, t):
    """
    Calculates the Root Mean Square Error.

    Args:
        y: Predicted values (Array of shape (N,))
        t: Target values (Array of shape (N,))

    Returns:
        Scalar float representing the error.
    """
```

```python
    # 1. Calculate Residuals (Vector Subtraction)
    residuals = y - t

    # 2. Square the Residuals (Element-wise Power)
    squared_error = residuals ** 2

    # 3. Compute the Mean (Sum / N)
    mean_squared_error = np.mean(squared_error)

    # 4. Take the Square Root
    root_mean_squared_error = np.sqrt(mean_squared_error)

    return root_mean_squared_error
```

Listing 2: Implementing RMSE

Order of Operations: NumPy executes this calculation in a specific sequence of array operations:

1. `y - t`: Creates a temporary array of differences.
2. `** 2`: Squares every element in that temporary array.
3. `np.mean`: Sums all elements and divides by the length N.
4. `np.sqrt`: Returns the scalar result.

1.3.3 Comparing Train and Validation RMSE

Because RMSE averages the error (divides by N), we can validly compare the error on the Training set (e.g., $N = 570$) with the error on the Validation set (e.g., $N = 122$).

2 Overfitting and Underfitting

We want our model to perform well on new, unseen data (the Test set). However, we face a Goldilocks problem.

- If the model is too simple, it cannot capture the underlying pattern (Underfitting).
- If the model is too complex, it captures the noise along with the pattern (Overfitting).

This tension is formally known as the **Bias-Variance Tradeoff**. In this section, we will visualize this phenomenon by observing how the polynomial degree M affects the shape of the curve.

2.1 The Bias-Variance Tradeoff

2.1.1 Underfitting (High Bias)

Consider the case where we choose a very low polynomial degree, such as $M = 0$ (constant) or $M = 1$ (linear).

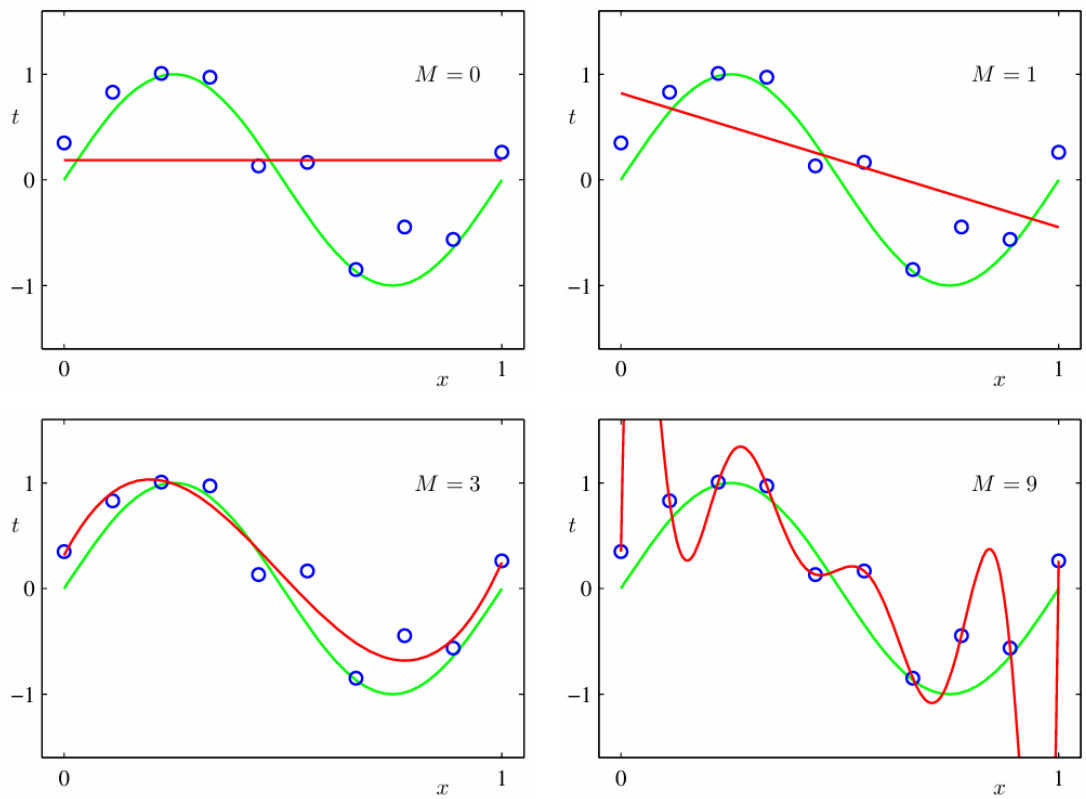

Figure 1: Plots of polynomials having various orders M, shown as red curves, fitted to the dataset shown in Part 1.

Visual Observation: Looking at the plots for $M = 0$ and $M = 1$, we see that the red line (the model) fails to pass near many of the blue data points.

- $M = 0$**:** The model is a flat horizontal line. It predicts the average value for everyone.
- $M = 1$**:** The model is a straight diagonal line.

Diagnosis: The model is **too rigid**. It has a strong "preconception" (Bias) that the relationship is linear. No matter how much data we give it, a straight line can never bend to fit a sine wave or the accelerating CO_2 curve.

- **High Bias:** The error comes from erroneous assumptions in the learning algorithm.
- **Symptoms:** High Training Error, High Validation Error.

2.1.2 Overfitting (High Variance)

Now consider the extreme opposite: a high-degree polynomial, such as $M = 9$ for a dataset with only $N = 10$ points.

Visual Observation: The red curve passes exactly through every single blue training point. The Training Error is effectively zero ($E_{train} \approx 0$). However, between the data points, the curve behaves wildly. It swings up and down in massive oscillations.

Diagnosis: The model is **too flexible**. It has so many degrees of freedom (parameters) that it can twist itself to memorize the specific noise in the training set.

- **High Variance:** The model is extremely sensitive to small fluctuations in the training data. If we moved one data point slightly, the entire curve would change drastically.
- **Symptoms:** Zero Training Error, Massive Validation Error.

2.1.3 The Sweet Spot

Our goal is to find the degree M that minimizes the **Total Expected Error**:

$$\text{Total Error} = \text{Bias}^2 + \text{Variance} + \text{Irreducible Noise} \tag{3}$$

- As M increases, Bias decreases (the model becomes more flexible).
- As M increases, Variance increases (the model becomes more unstable).

2.2 The Phenomenon of Exploding Coefficients

In Section 2.1, we observed visually that high-order polynomials ($M = 9$) produce wild oscillations, passing perfectly through training points but swinging violently between them. To understand the mechanics of this failure, we must look "under the hood" at the learned parameters $\mathbf{w}$.

2.2.1 Examining the Weight Table

Let us compare the coefficients learned for the synthetic sine wave dataset for different polynomial degrees. The table below summarizes the values of $\mathbf{w}^\star$ obtained by minimizing the sum of squared errors.

Coefficient	$M = 0$	$M = 1$	$M = 6$	$M = 9$
w_0 (Bias)	0.19	0.82	0.31	0.35
w_1 (x)	-	-1.27	7.99	232.37
w_2 (x^2)	-	-	-25.43	-5321.83
w_3 (x^3)	-	-	17.37	48568.31
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
w_9 (x^9)	-	-	-	**125201.43**

Table 1: The growth of coefficients as model complexity increases (Values from Bishop, 2006).

Observation:

- For $M = 0$ **and** $M = 1$, the coefficients are small, interpretable numbers (roughly in the range $[-1, 1]$).
- For $M = 9$, the coefficients explode. w_9 is over $125,000$, and other terms reach similarly massive magnitudes with alternating signs.

2.2.2 The Mechanism of Cancellation

How can a model with a coefficient of $125,201$ produce a prediction like $t \approx 0.5$? The answer lies in **destructive interference** (cancellation).

The prediction is the sum:

$$y(x) = w_0 + w_1 x + \cdots + w_9 x^9 \tag{4}$$

For a specific training point x_n, the term $w_9 x_n^9$ might equal $+1,000,000$. The term $w_8 x_n^8$ might equal $-999,999.5$. These massive numbers cancel each other out almost perfectly, leaving a remainder of 0.5, which matches the target t_n.

Why does the model do this? The model is trying to minimize the error to *zero*. To hit every single noisy data point exactly, the polynomial needs to execute extremely sharp turns. A sharp turn implies a massive derivative (slope).

- To get a massive slope, you need massive coefficients.
- To ensure the curve doesn't stay massive (and miss the next point), the signs must alternate $(+-+-)$ so the terms cancel out exactly at the locations of the training data.

2.2.3 Generalization Failure

While this delicate balancing act works perfectly at the specific coordinates of the training data x_{train}, it is incredibly fragile. If we move slightly away from a training point to a test point x_{test}, the perfect cancellation breaks down. The term $+1,000,000$ is no longer balanced by $-999,999.5$. The residual difference dominates the prediction, causing the value $y(x)$ to shoot off to infinity or negative infinity.

This gives us a crucial diagnostic rule for machine learning:

> **Diagnostic Rule:** If you see your model weights growing to unusually large numbers (relative to the scale of your data), you are almost certainly overfitting.

This observation leads directly to the solution: if large weights are the problem, we should mathematically penalize them. This is the foundation of **Regularization** (Ridge Regression) (Krogh & Hertz, 1992).

2.3 Hyperparameter Tuning

We have identified that the polynomial degree M controls the complexity of the model.

- M too low $\rightarrow$ Underfitting (Bias).
- M too high $\rightarrow$ Overfitting (Variance).

But how do we find the "just right" M? Unlike the weights $\mathbf{w}$, which are learned automatically by the Normal Equation, the degree M is not part of the equation itself. It determines the *structure* of the equation.

2.3.1 Parameters vs. Hyperparameters

In machine learning, we distinguish between two types of variables:

1. **Parameters (w):** Internal variables learned from the training data (e.g., the coefficients w_0, w_1).
2. **Hyperparameters (M):** External configuration variables chosen by the data scientist *before* training begins. They control the learning process itself.

Since there is no closed-form formula to calculate the optimal M, we must use an iterative search strategy. The simplest and most common method is **Grid Search** (brute-force loop) (Bergstra et al., 2011).

2.3.2 The Training Loop

We will systematically try every integer degree from $M = 0$ to $M = 20$. For each candidate degree, we perform a complete "Train-Evaluate" cycle.

```python
# Initialize lists to store results
degrees = range(0, 21)  # Try M = 0 .. 20
train_rmse = []
val_rmse = []
models = {} # Store the learned weights for later

for M in degrees:
    # 1. Feature Engineering (Construct Phi matrices)
    # Note: We create a NEW matrix for every M
    Phi_train = design_matrix(x_train, M)
    Phi_val   = design_matrix(x_val, M)

    # 2. Training (Fit on TRAIN set only)
    # This finds the optimal w for this specific M
    w = fit_least_squares(Phi_train, t_train)
    models[M] = w

    # 3. Prediction
    # We predict on BOTH sets to compare performance
    y_train = Phi_train @ w
    y_val   = Phi_val @ w

    # 4. Evaluation (Calculate RMSE)
    e_train = rmse(y_train, t_train)
    e_val   = rmse(y_val, t_val)

    # 5. Store results
    train_rmse.append(e_train)
    val_rmse.append(e_val)

    print(f"Degree {M}: Train RMSE = {e_train:.3f}, Val RMSE = {e_val:.3f}"
        )
```

Listing 3: The Hyperparameter Tuning Loop

Step 1 - Dynamic Matrix Construction: Inside the loop, `design_matrix(x, M)` is called repeatedly.

- When $M = 1$, Φ has 2 columns (Bias, Linear).
- When $M = 20$, Φ has 21 columns.

The shape of our data changes in every iteration. This is why we cannot compute Φ once outside the loop.

Step 2 - Fitting vs. Evaluating: Notice the asymmetry in the arguments:

- `fit_least_squares(Phi_train, t_train)`: The weights $\mathbf{w}$ are calculated minimizing error on the **Training** set. The Validation set is completely invisible to this step.

Step 3 - The Moment of Truth:

- `y_val = Phi_val @ w`: We take the weights learned from the past (Training) and apply them to the recent past (Validation).
- This simulates the real-world constraint: "If we had used this model in the past, how well would it have predicted the subsequent years?"

2.3.3 Selecting the Best Model

Once the loop finishes, we have a list of Validation RMSEs. The optimal hyperparameter M^* is simply the one that resulted in the lowest error on the validation set.

$$M^* = \underset{M}{\operatorname{argmin}}(\text{RMSE}_{val}) \tag{5}$$

```
best_M = np.argmin(val_rmse)
print(f"Best Degree: {best_M}")
```

3 The Final Model

Let us assume we have identified $M^* = 2$ (a quadratic model) as the best trade-off between bias and variance. However, simply taking the model trained on the initial 70% of the data would be wasteful. We have another 15% of data (the Validation set) that we have "seen" but haven't used to train the weights $\mathbf{w}$. We should use every available scrap of information to refine our parameter estimates before deploying the model.

3.1 Merging Train and Validation

Once the hyperparameter M^* is fixed, the distinction between "Training" and "Validation" becomes obsolete. They are both simply "past data."

By merging them, we increase our effective sample size N.

- **Statistical Benefit:** The variance of our parameter estimates $\mathbf{w}$ is inversely proportional to N. By increasing N (from 70% to 85%), we reduce the uncertainty in our model (Halevy et al., 2009).

- **Practical Benefit:** The model has seen more recent data (the validation years), making it better equipped to predict the future (the test years).

3.1.1 Python Implementation

We use NumPy to stitch the arrays together.

```
# 1. Merge Training and Validation Sets
x_combined = np.concatenate((x_train, x_val))
t_combined = np.concatenate((t_train, t_val))

# 2. Re-Standardize
# CRITICAL: We must calculate new mean/std based on this larger set
mu_final = x_combined.mean()
sig_final = x_combined.std()

x_combined_z = (x_combined - mu_final) / sig_final
x_test_z     = (x_test - mu_final) / sig_final  # Apply to Test Set

# 3. Build Design Matrices with Best Degree (e.g., M=2)
M_star = 2
Phi_combined = design_matrix(x_combined_z, M_star)
Phi_test     = design_matrix(x_test_z, M_star)

# 4. Final Fit
w_final = fit_least_squares(Phi_combined, t_combined)
```

Listing 4: Merging and Refitting

3.1.2 Blind Evaluation on the Test Set

This is the most solemn moment in the machine learning process. The Test Set (x_{test}, t_{test}) has been locked in a vault until now. We have never used it to train weights, nor have we used it to choose M. We now run the final prediction:

```
# Predict on Test Set
y_test = Phi_test @ w_final

# Calculate Final RMSE
test_rmse = rmse(y_test, t_test)
print(f"Final Test RMSE: {test_rmse:.3f}")
```

Listing 5: Final Evaluation

The One-Shot Rule: This evaluation allows **no do-overs**.

- If the Test RMSE is bad, you cannot go back and change M to 3.
- If you do go back, the Test Set effectively becomes a Validation Set, and you have no data left to truthfully report generalization performance.
- This number is your final report card. It represents the unbiased estimate of how the model will perform on completely new data in the real world (Russell & Norvig, 2010).

3.2 Interpretation of Mauna Loa Results

Having trained our final quadratic model ($M = 2$) and evaluated it on the test set, we must now interpret the results. A low RMSE is desirable, but in science, we must also inspect the **Residuals**, to ensure they look like random noise.

3.2.1 Trend vs. Seasonality

Let us examine the plot of the final fit.

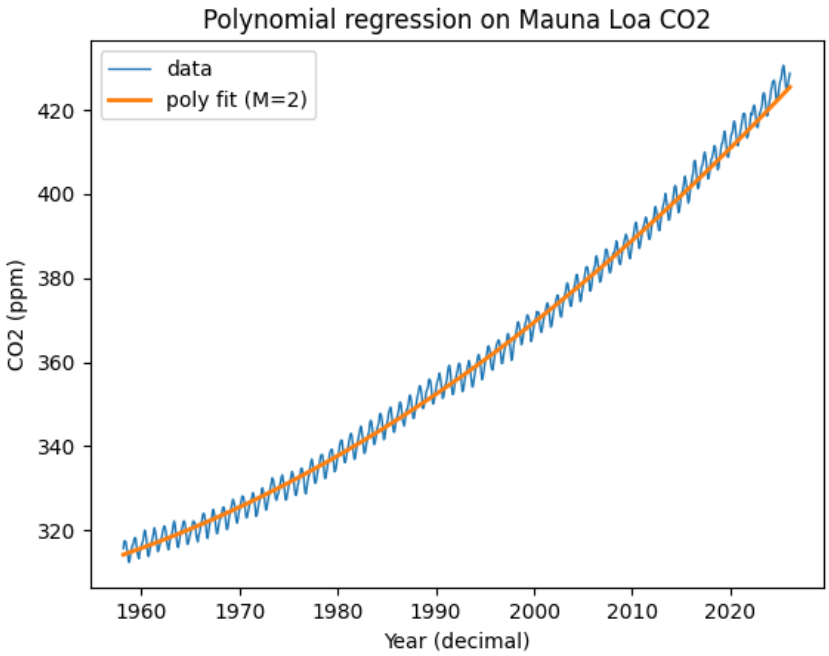

Figure 2: Visualization of the final quadratic model ($M = 2$) applied to the Mauna Loa dataset. The orange curve represents the model's predictions, capturing the long-term accelerating trend of atmospheric CO_2 but failing to model the high-frequency seasonal oscillations (the blue "sawtooth" pattern).

1. **The Trend (Success):** The model successfully captures the long-term trajectory of CO_2 concentration. It correctly identifies that the rise is not linear ($M = 1$) but accelerating ($M = 2$). The quadratic term w_2x^2 has a positive coefficient, reflecting the increasing rate of fossil fuel emissions over the decades.

2. **The Seasonality (Failure):** The data points oscillate around the trend line in a regular "sawtooth" pattern, rising in winter and falling in summer. Our model drives straight through the middle of these oscillations.

Physical Context: This oscillation is the "breathing" of the Earth. The majority of the planet's landmass is in the Northern Hemisphere. In spring/summer, vegetation grows and absorbs CO_2 (level drops). In autumn/winter, vegetation decays and releases CO_2 (level rises). This cycle repeats every 12 months (Keeling et al., 1976).

3.2.2 Residual Analysis

Ideally, the residuals $e_n = y_n - t_n$ should be **White Noise**—random, uncorrelated errors with no discernible pattern. This would indicate that the model has extracted *all* predictable information from the signal.

In our case, if we plot the residuals over time, we would see a perfect sine wave.

> **Diagnostic Rule:** If your residuals have a shape (structure), your model is underfitting. You are leaving information on the table.

Our quadratic model has captured the *climate* (long-term trend) but missed the *weather* (seasonal variation).

3.2.3 The Mathematical Limitation of Polynomials

Why did our model fail to learn the seasonality? We gave it the best algorithm (Least Squares) and the best data (Mauna Loa). The failure is in the **Hypothesis Space**. We restricted our model to be a linear combination of monotonic powers: $1, x, x^2$.

Global vs. Local Behavior

- **Polynomials:** Functions like x^2 or x^{10} are **unbounded**. As $x \to \infty$, $|y| \to \infty$. They cannot oscillate forever; they must eventually shoot off to infinity.
- **Periodic Data:** Functions like $\sin(x)$ are **bounded**. They repeat indefinitely within a fixed range $[-1, 1]$.

A polynomial can approximate a sine wave locally (via a Taylor Series expansion), but it requires an infinite number of terms to approximate a sine wave globally. Since we limited ourselves to $M = 2$, the model "chose" to ignore the sine wave entirely rather than trying to chase it and ruining the long-term trend (as we saw with the exploding coefficients in $M = 9$).

3.2.4 The Power of Feature Engineering

This limitation reveals the true power of the **Design Matrix**. Recall that our model is:

$$y(x) = \mathbf{w}^T \boldsymbol{\phi}(x) \tag{6}$$

We chose $\boldsymbol{\phi}(x) = [1, x, x^2]$. However, nothing stops us from adding *domain knowledge* to our features. If we know the data is annual, we could construct a design matrix with trigonometric features:

$$\boldsymbol{\phi}_{advanced}(x) = \begin{bmatrix} 1 \\ x \\ x^2 \\ \sin(2\pi x) \\ \cos(2\pi x) \end{bmatrix} \tag{7}$$

If we fed this matrix into the *exact same* Least Squares algorithm:

- The solver would assign weights to x^2 to fit the trend.

- The solver would assign weights to $\sin(2\pi x)$ to fit the seasons.

This demonstrates that linear regression is not limited to straight lines; it is limited only by our creativity in choosing the basis functions $\phi(x)$.

4 Regularization (Ridge Regression)

If the symptom of overfitting is "large weights," then the cure is simple: **we must forbid the weights from getting too large.**

4.1 The Cure for Overfitting

We achieve this by modifying our objective. Instead of merely minimizing the prediction error (SSE), we minimize the error *plus* a penalty for complexity.

4.1.1 The Regularized Error Function

We define a new loss function $E_{ridge}(\mathbf{w})$:

$$E_{ridge}(\mathbf{w}) = \underbrace{\frac{1}{2}||\Phi\mathbf{w} - \mathbf{t}||^2}_{\text{Data Term (Fit)}} + \underbrace{\frac{\lambda}{2}||\mathbf{w}||^2}_{\text{Regularization Term (Penalty)}} \tag{8}$$

Here, $||\mathbf{w}||^2 = w_0^2 + w_1^2 + \cdots + w_M^2$ is the squared Euclidean length (magnitude) of the weight vector.

The Hyperparameter λ (Lambda): This new term introduces a crucial hyperparameter, $\lambda \geq 0$, which controls the strength of the penalty.

- **If $\lambda = 0$:** The penalty vanishes. We are back to standard Least Squares (OLS).
- **If $\lambda \to \infty$:** The penalty dominates. The only way to minimize the cost is to set all weights $\mathbf{w}$ to zero. The model becomes a flat line at $y = 0$.
- **The Sweet Spot:** An intermediate λ forces the model to balance fitting the data (low SSE) with keeping the weights small (low variance).

This specific form of regularization is known as **Ridge Regression** or L_2 **Regularization** (Hoerl & Kennard, 1970).

4.1.2 Deriving the Closed-Form Ridge Solution

Just like with Ordinary Least Squares, we can find the optimal weights $\mathbf{w}_{ridge}$ analytically by taking the gradient and setting it to zero.

Step 1: The Gradient We differentiate E_{ridge} with respect to $\mathbf{w}$:

$$\nabla E_{ridge} = \nabla \left(\frac{1}{2} ||\Phi \mathbf{w} - \mathbf{t}||^2 \right) + \nabla \left(\frac{\lambda}{2} \mathbf{w}^T \mathbf{w} \right) \tag{9}$$

We already derived the gradient of the first term (the SSE) in Section 4.2:

$$\Phi^T \Phi \mathbf{w} - \Phi^T \mathbf{t}$$

Now we need the gradient of the penalty term $\frac{\lambda}{2} \mathbf{w}^T \mathbf{w}$. Using the rule $\nabla(\mathbf{x}^T \mathbf{x}) = 2\mathbf{x}$:

$$\nabla \left(\frac{\lambda}{2} \mathbf{w}^T \mathbf{w} \right) = \frac{\lambda}{2}(2\mathbf{w}) = \lambda \mathbf{w}$$

Adding them together gives the total gradient:

$$\nabla E_{ridge} = (\Phi^T \Phi \mathbf{w} - \Phi^T \mathbf{t}) + \lambda \mathbf{w} \tag{10}$$

Step 2: Solving for Zero Set the gradient to zero:

$$\Phi^T \Phi \mathbf{w} + \lambda \mathbf{w} = \Phi^T \mathbf{t} \tag{11}$$

We can factor out $\mathbf{w}$ on the left side. However, $\Phi^T \Phi$ is a matrix and λ is a scalar. To add them, we must multiply λ by the **Identity Matrix** I (a matrix with 1s on the diagonal and 0s elsewhere).

$$(\Phi^T \Phi + \lambda I)\mathbf{w} = \Phi^T \mathbf{t} \tag{12}$$

Step 3: The Solution Assuming the matrix in parentheses is invertible, we multiply by its inverse:

$$\mathbf{w}_{ridge} = (\Phi^T \Phi + \lambda I)^{-1} \Phi^T \mathbf{t} \tag{13}$$

4.1.3 Why this works (Numerical Stability)

Previously, we learned that $\Phi^T \Phi$ can be "ill-conditioned" (singular), causing the inverse to explode. By adding λI, we are adding a positive number λ to every element on the main diagonal of $\Phi^T \Phi$.

Mathematically, this shifts the eigenvalues:

$$\text{Eigenvalues}(\Phi^T \Phi + \lambda I) = \text{Eigenvalues}(\Phi^T \Phi) + \lambda$$

Even if $\Phi^T \Phi$ has an eigenvalue of 0 (making it non-invertible), the new matrix has an eigenvalue of $0 + \lambda > 0$. Therefore, Ridge Regression is **always** numerically stable. The matrix $(\Phi^T \Phi + \lambda I)$ is always invertible, even if we have more features than data points (Ng, 2004).

4.2 Implementation of Ridge Regression

We have previously derived the closed-form solution for the regularized weights:

$$\mathbf{w}_{ridge} = (\Phi^T \Phi + \lambda I)^{-1} \Phi^T \mathbf{t} \tag{14}$$

To implement this, we need to construct the matrix $\Phi^T \Phi$, add the regularization term λI, and then solve for $\mathbf{w}$.

4.2.1 Constructing the Identity Matrix

In Linear Algebra, multiplying by the identity matrix I is equivalent to multiplying by 1. However, adding λI to a matrix adds the value λ to every diagonal element. In NumPy, we generate this matrix using the function `np.eye` (a pun on "Identity" → "I" → "Eye").

4.2.2 The Python Code

We define a function `fit_ridge` that takes the Design Matrix, the target vector, and the scalar hyperparameter λ.

```python
import numpy as np

def fit_ridge(Phi, t, lam):
    """
    Solves the Ridge Regression system: (Phi^T*Phi + lam*I)w = Phi^T*t

    Args:
        Phi: Design Matrix of shape (N, M+1)
        t: Target vector of shape (N,)
        lam: Regularization strength (scalar float)

    Returns:
        w: Optimal weights vector
    """
    # 1. Get the number of features (M+1)
    M1 = Phi.shape[1]

    # 2. Construct the Identity Matrix
    I = np.eye(M1)

    # Optional: Do not penalize the bias term (w_0)
    # The bias determines the vertical offset (mean) of the data.
    # We usually don't want to shrink the mean towards zero.
    I[0, 0] = 0

    # 3. Construct the "A" matrix (LHS of equation)
    # A = Phi^T * Phi + lambda * I
    A = (Phi.T @ Phi) + (lam * I)

    # 4. Construct the "b" vector (RHS of equation)
    # b = Phi^T * t
    b = Phi.T @ t

    # 5. Solve the linear system A * w = b
    # Since A is square and symmetric positive-definite (thanks to lambda),
    # we can use the standard solver.
    w = np.linalg.solve(A, b)

    return w
```

Listing 6: Ridge Regression Implementation

Matrix Addition (`lam * I`)" Here we see the "broadcasting" of the scalar λ.

- `np.eye(M1)` creates a matrix of 0s and 1s.
- `lam * np.eye(M1)` creates a matrix where the diagonal elements are all equal to λ.
- Adding this to `Phi.T @ Phi` boosts the diagonal, ensuring numerical stability.

`np.linalg.solve` vs `np.linalg.inv`: Notice we use `np.linalg.solve(A, b)` instead of `np.linalg.inv(A) @ b`.

- **Efficiency:** Calculating the inverse explicitly is computationally expensive ($O(N^3)$). Solving the system $Ax = b$ directly is faster.
- **Stability:** While Ridge Regression guarantees that A is invertible (because we added λ), using `solve` is still numerically more precise than multiplying by an explicit inverse (Golub & Van Loan, 2013).

4.2.3 Finding the Optimal λ

Just as we searched for the best polynomial degree M, we must now search for the best regularization strength λ. However, unlike M (which is an integer $1, 2, 3 \ldots$), λ is a continuous number that spans many orders of magnitude. A λ of 0.01 has a very different effect from 100. Therefore, we search on a **Logarithmic Scale**.

```python
# Create a range of lambdas: 10^-6, 10^-5, ..., 10^5, 10^6
lambdas = np.logspace(-6, 6, 13)

best_rmse = float('inf')
best_lam = None

for lam in lambdas:
    # Train Ridge Model
    w_ridge = fit_ridge(Phi_train, t_train, lam)

    # Validate
    y_val = Phi_val @ w_ridge
    err = rmse(y_val, t_val)

    print(f"Lambda {lam:.1e}: Val RMSE {err:.3f}")

    if err < best_rmse:
        best_rmse = err
        best_lam = lam

print(f"Best Ridge Lambda: {best_lam}")
```

Listing 7: Grid Search for Lambda

Why `logspace`? Using `np.linspace` (linear spacing) would check $0.1, 1.1, 2.1 \ldots$. It would miss the tiny values like 10^{-5} entirely. `np.logspace(-6, 6, 13)` generates $[10^{-6}, 10^{-5}, \ldots, 10^{6}]$. This allows us to scan the "Magnitude" of the penalty rather than specific values.

4.2.4 The Result

When applied to the Mauna Loa dataset (specifically with a higher degree model like $M = 9$), Ridge Regression typically selects a moderate λ (e.g., 0.1). This allows us to keep the flexibility of the high-degree polynomial (to fit the trend) but suppresses the exploding coefficients.

Appendices

A Detailed Matrix Calculus Derivations

In Section 4, we used two key identities to derive the Normal Equation:

1. $\nabla_{\mathbf{w}}(\mathbf{w}^T\mathbf{t}) = \mathbf{t}$
2. $\nabla_{\mathbf{w}}(\mathbf{w}^T A\mathbf{w}) = 2A\mathbf{w}$ (if A is symmetric)

To prove these, we must look inside the vectors and matrices at the individual scalar components.

A.1 A.1 The Gradient Defined

The gradient of a scalar function $f(\mathbf{w})$ with respect to a vector $\mathbf{w} \in \mathbb{R}^M$ is defined as the vector of partial derivatives:

$$\nabla_{\mathbf{w}} f = \begin{pmatrix} \frac{\partial f}{\partial w_1} \\ \vdots \\ \frac{\partial f}{\partial w_M} \end{pmatrix} \tag{15}$$

To find the gradient vector, we pick an arbitrary index k (where $1 \leq k \leq M$), calculate the partial derivative $\frac{\partial f}{\partial w_k}$, and then stack the results.

A.2 A.2 Gradient of a Linear Form

Let $f(\mathbf{w}) = \mathbf{w}^T\mathbf{t}$, where $\mathbf{t}$ is a constant vector. In component form (summation notation), this dot product is:

$$f(\mathbf{w}) = \sum_{i=1}^{M} w_i t_i \tag{16}$$

Now, take the partial derivative with respect to a specific weight w_k:

$$\frac{\partial f}{\partial w_k} = \frac{\partial}{\partial w_k}(w_1 t_1 + \cdots + w_k t_k + \cdots + w_M t_M) \tag{17}$$

Since w_i and w_k are independent variables (for $i \neq k$), the derivative of any term where $i \neq k$ is zero. The only term that survives is $w_k t_k$.

$$\frac{\partial}{\partial w_k}(w_k t_k) = t_k \tag{18}$$

So, the k-th component of the gradient vector is t_k. Stacking these gives:

$$\nabla_{\mathbf{w}}(\mathbf{w}^T\mathbf{t}) = \mathbf{t} \tag{19}$$

A.3 A.3 Gradient of a Quadratic Form

This is the most critical derivation for Least Squares. Let $f(\mathbf{w}) = \mathbf{w}^T A\mathbf{w}$, where A is an $M \times M$ matrix. In summation notation, this double dot product is:

$$f(\mathbf{w}) = \sum_{i=1}^{M}\sum_{j=1}^{M} w_i A_{ij} w_j \tag{20}$$

We want to compute $\frac{\partial f}{\partial w_k}$. We must be careful because w_k appears twice in the sum: once as w_i (when $i = k$) and once as w_j (when $j = k$).

We split the sum into three cases based on the indices i and j:

1. **Case 1 ($i = k, j \neq k$):** The row is k.
$$w_k A_{kj} w_j$$
Derivative w.r.t w_k: $A_{kj} w_j$.

2. **Case 2 ($i \neq k, j = k$):** The column is k.
$$w_i A_{ik} w_k$$
Derivative w.r.t w_k: $w_i A_{ik}$.

3. **Case 3 ($i = k, j = k$):** The diagonal term.
$$w_k A_{kk} w_k = A_{kk} w_k^2$$
Derivative w.r.t w_k: $2A_{kk} w_k$.

A.3.1 Summing the Derivatives

Combining these contributions over all i and j:

$$\frac{\partial f}{\partial w_k} = \sum_{j \neq k} A_{kj} w_j + \sum_{i \neq k} w_i A_{ik} + 2A_{kk} w_k \tag{21}$$

We can merge the diagonal term ($2A_{kk} w_k$) back into the sums by splitting it into $A_{kk} w_k + A_{kk} w_k$. One part completes the first sum (over j), and the other completes the second sum (over i).

$$\frac{\partial f}{\partial w_k} = \sum_{j=1}^{M} A_{kj} w_j + \sum_{i=1}^{M} w_i A_{ik} \tag{22}$$

Let us look at what these sums represent:

- $\sum_j A_{kj} w_j$ is the dot product of the k-th **row** of A with $\mathbf{w}$. This is the k-th element of the vector $A\mathbf{w}$.
- $\sum_i w_i A_{ik} = \sum_i A_{ik} w_i$ is the dot product of the k-th **column** of A with $\mathbf{w}$. This is the k-th element of the vector $A^T\mathbf{w}$.

Thus, the gradient vector is:

$$\nabla_{\mathbf{w}}(\mathbf{w}^T A \mathbf{w}) = A\mathbf{w} + A^T\mathbf{w} = (A + A^T)\mathbf{w} \tag{23}$$

A.3.2 Symmetry Simplification

In the specific case of Least Squares, $A = \Phi^T \Phi$. A matrix multiplied by its own transpose is always **Symmetric** ($A = A^T$). $(B^T B)^T = B^T (B^T)^T = B^T B$.

Since $A = A^T$, the terms $A\mathbf{w}$ and $A^T\mathbf{w}$ are identical.

$$\nabla_{\mathbf{w}}(\mathbf{w}^T A \mathbf{w}) = A\mathbf{w} + A\mathbf{w} = 2A\mathbf{w} \tag{24}$$

B PythonCheat Sheet for Linear Algebra

In scientific computing, the translation from mathematical blackboard to Python code is often where bugs are introduced. This appendix summarizes the essential NumPy operations used in this textbook, mapping specific Linear Algebra concepts to their code equivalents.

B.1 B.1 Vectors and Shapes

The most common error in NumPy is a **Shape Mismatch**. Mathematical vectors are strictly column vectors ($N \times 1$) or row vectors ($1 \times N$). NumPy, however, has "Rank-1 Arrays" which have no orientation.

Math Concept	Notation	NumPy Code	Shape / Note
Scalar	s	`s = 1.5`	`()` (0-D)
Flat Array	-	`a = np.array([1, 2])`	`(2,)` **Danger Zone**. neither row nor column.
Column Vector	$\mathbf{v} \in \mathbb{R}^N$	`v = a.reshape(-1, 1)`	`(N, 1)` Standard for Linear Algebra.
Row Vector	$\mathbf{v}^T$	`vt = a.reshape(1, -1)`	`(1, N)`

Table 2: Data Structures in NumPy

Reshaping Tricks

- `-1` in `reshape` means "infer this dimension."
- `x[:, np.newaxis]` is a shorthand to convert a flat array `(N,)` into a column vector `(N, 1)`.

B.2 Matrix Operations

B.2.1 Multiplication: Element-wise vs. Dot Product

In mathematics, "multiplication" usually implies the dot product or matrix multiplication. In Python, the asterisk * usually implies element-wise multiplication.

```python
import numpy as np
A = np.array([[1, 2], [3, 4]])
B = np.array([[2, 0], [1, 2]])

# 1. Element-wise Multiplication (Hadamard Product)
# Result: [[2, 0], [3, 8]]
# Math: A_ij * B_ij
C_element = A * B

# 2. Matrix Multiplication (Dot Product)
# Result: [[4, 4], [10, 8]]
# Math: Sum(A_ik * B_kj)
C_dot = A @ B  # The '@' operator (Preferred)
C_dot = np.dot(A, B) # (Legacy syntax)
```

Listing 8: Matrix Multiplication Types

B.2.2 Transpose and Identity

```
# Transpose (Math: A^T)
A_T = A.T

# Identity Matrix (Math: I)
# Creates a 3x3 square matrix with 1s on diagonal
I = np.eye(3)
```

B.3 Solving Linear Systems

We almost never compute the inverse A^{-1} explicitly. We use solvers.

Problem	Math	NumPy Function	Use Case
Inverse	A^{-1}	`np.linalg.inv(A)`	**Avoid.** Unstable and slow.
Linear System	$x = A^{-1}b$	`np.linalg.solve(A, b)`	Use for Ridge Regression (A is square).
Least Squares	$\min \|\|Ax - b\|\|^2$	`np.linalg.lstsq(A, b)`	Use for Polynomial Regression (A is non-square).
Pseudo-Inverse	A^+	`np.linalg.pinv(A)`	Use for theoretical analysis.

Table 3: Solvers in `numpy.linalg`

B.4 Norms and Statistics

B.4.1 Euclidean Norm (L2 Norm)

Used in the Ridge Regression penalty $||\mathbf{w}||^2$.

```
w = np.array([3, 4])

# Manual Calculation
norm_sq = w.T @ w  # Result: 25

# NumPy Built-in
norm = np.linalg.norm(w) # Result: 5.0 (Sqrt of 25)
```

B.4.2 Broadcasting (The "Magic" of NumPy)

Broadcasting allows operations on arrays of different shapes. This was crucial in Standardization.

```
X = np.array([[10, 20], [30, 40]]) # Shape (2, 2)
mu = np.array([1, 2])              # Shape (2,)

# Math: Subtract mu from EVERY row of X.
# NumPy automatically "stretches" mu to match X.
X_centered = X - mu
# Result: [[9, 18], [29, 38]]
```

Rule of Broadcasting: Two dimensions are compatible when: 1. They are equal, or 2. One of them is 1.

B.5 Polynomial Specifics

Constructing the Vandermonde Matrix (Design Matrix) without loops.

```
x = np.array([1, 2, 3])
M = 2

# Using vander (Note: NumPy's vander returns decreasing powers x^M ... x^0)
Phi_flipped = np.vander(x, M+1)

# Using our manual method (Increasing powers x^0 ... x^M)
Phi = np.vstack([x**j for j in range(M+1)]).T
```

References

Bergstra, J., Bardenet, R., Bengio, Y., & Kégl, B. (2011). Algorithms for hyper-parameter optimization. In *Advances in neural information processing systems* (Vol. 24). (Discusses strategies for selecting hyperparameters, contrasting Grid Search with Random Search and Bayesian Optimization.)

Golub, G. H., & Van Loan, C. F. (2013). *Matrix computations* (4th ed.). Johns Hopkins University Press. (The bible of numerical linear algebra, detailing the properties and stability issues of Vandermonde matrices.)

Halevy, A., Norvig, P., & Pereira, F. (2009). The unreasonable effectiveness of data. *IEEE Intelligent Systems*, *24*(2), 8–12. (Argues that adding more data (merging train+val) is often more effective than improving algorithms.)

Hoerl, A. E., & Kennard, R. W. (1970). Ridge regression: Biased estimation for nonorthogonal problems. *Technometrics*, *12*(1), 55–67. (The foundational paper introducing the addition of λI to the correlation matrix.)

Hyndman, R. J., & Athanasopoulos, G. (2018). *Forecasting: Principles and practice* (2nd ed.). Melbourne, Australia: OTexts. (The standard reference for time series methodology, explicitly warning against random shuffling.)

Keeling, C. D., Bacastow, R. B., Bainbridge, A. E., Ekdahl, C. A., Guenther, P. R., Waterman, L. S., & Chin, J. F. S. (1976). Atmospheric carbon dioxide variations at mauna loa observatory, hawaii. *Tellus*, *28*(6), 538–551.

Krogh, A., & Hertz, J. A. (1992). A simple weight decay can improve generalization. In *Advances in neural information processing systems* (pp. 950–957). (Demonstrates mathematically why constraining the magnitude of weights (weight decay) reduces generalization error.)

Ng, A. Y. (2004). Feature selection, l1 vs. l2 regularization, and rotational invariance. In *Proceedings of the twenty-first international conference on machine learning* (p. 78). (Explains why L2 regularization (Ridge) is numerically stable and rotationally invariant.)

Russell, S. J., & Norvig, P. (2010). *Artificial intelligence: A modern approach* (3rd ed.). Pearson Education. (The standard textbook for AI, detailing the strict separation of Test sets.)

Vapnik, V. N. (1998). *Statistical learning theory*. New York: Wiley. (The foundational text defining the mathematical difference between empirical risk (training error) and true risk (test error).)